

Kubernetes Hardening Checklist

Detailed checklist for secure Kubernetes workload design, pull request review, validation evidence, and CIS Benchmark alignment.

Field	Value
Document Type	Checklist
Author	Jagriti Banerjee
Project Scope	Private Kubernetes DevSecOps lab using Docker, Kind, Calico, Trivy, Falco, OPA/Gatekeeper, ArgoCD, and kube-bench
Audience	Security engineers, DevSecOps reviewers, recruiters, and technical interviewers
Classification	Private lab / portfolio evidence

Reviewer snapshot

Review Dimension	Included in Document
Technical depth	Control rationale, validation commands, expected results, risk interpretation, and remediation guidance.
Evidence model	Screenshot references, manifest fields, scanner outputs, and repeatable kubectl commands.
Professional use	Can be used as a GitHub portfolio artifact, interview talking point, and internal-style security record.
CIS alignment	Includes a dedicated CIS Kubernetes Benchmark section with lab-context interpretation.

Scope statement

This document is based on controlled, authorized testing performed inside a private lab environment. It does not represent a production certification and does not include public or third-party targets. The focus is professional evidence, repeatable validation, and security engineering reasoning.

Spacing and readability note

The document is intentionally laid out with compact paragraphs, split tables, and dedicated evidence pages so that detail is preserved without crowded pages or large blank gaps.

Document coverage

Coverage Area	Included
Assessment narrative	Executive summary, scope, methodology, technical analysis, and validation interpretation.
Evidence references	Relevant screenshots, command outputs, reports, and expected retest outcomes.
Remediation planning	Priority-based actions, production notes, and follow-up recommendations.
Benchmark context	CIS Kubernetes Benchmark alignment and lab-specific interpretation.

1. Executive Summary

This checklist converts the project security implementation into a reusable Kubernetes hardening checklist. It is designed for technical reviewers who want to understand what was hardened, why it matters, how it was validated, and what evidence proves closure.

2. Checklist Scope

Control Family	Included Checks
Namespace	Dedicated namespace, labels, Pod Security restricted enforce/warn/audit modes.
Identity	Dedicated ServiceAccount, no default SA dependency, token automount disabled.
Runtime	Non-root user, fixed UID/GID, no privilege escalation, seccomp RuntimeDefault.
Filesystem	Read-only root filesystem and controlled writable /tmp emptyDir.
Linux capabilities	Drop ALL capabilities unless explicitly justified.
Reliability/security hygiene	Readiness/liveness probes and resource requests/limits.
Network	Default-deny ingress/egress and explicit allow rules.
Validation	Trivy config scan, kubectl jsonpath, NetworkPolicy test pods, kube-bench reference.

3. Detailed Hardening Checklist

ID	Control	Expected State	Result	Evidence
K8S-001	Namespace exists	devsecops namespace created	Pass	kubectl get ns devsecops
K8S-002	Restricted Pod Security	enforce/warn/audit restricted labels	Pass	kubectl get ns --show-labels
K8S-003	Dedicated ServiceAccount	demo-app-sa used by Deployment	Pass	deployment.yaml
K8S-004	Token automount disabled	automountServiceAccountToken false	Pass	deployment.yaml and rbac.yaml
K8S-005	Non-root execution	runAsNonRoot true and UID 10001	Pass	jsonpath output
K8S-006	No privilege escalation	allowPrivilegeEscalation false	Pass	securityContext
K8S-007	Capabilities dropped	capabilities.drop ALL	Pass	securityContext
K8S-008	Default seccomp profile	RuntimeDefault configured	Pass	pod securityContext
K8S-009	Read-only root filesystem	/app write fails	Pass	runtime exec test
K8S-010	Writable tmp isolated	/tmp write succeeds	Pass	emptyDir validation
K8S-011	Resource constraints	CPU and memory requests/limits set	Pass	deployment.yaml
K8S-012	Health probes	readiness and liveness probes configured	Pass	deployment.yaml
K8S-013	Default-deny NetworkPolicy	podSelector {} denies traffic by default	Pass	network-policy.yaml
K8S-014	Trusted ingress allowed	ingress namespace and approved pods allowed	Pass	curl validation
K8S-015	Secrets denied by RBAC	ServiceAccount cannot get secrets	Pass	kubectl auth can-i
K8S-016	Manifest scanning	Trivy config scan recorded	Pass	security/reports

4. Golden Manifest Snippet

```
spec:
  serviceAccountName: demo-app-sa
  automountServiceAccountToken: false
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001
  seccompProfile:
    type: RuntimeDefault
  containers:
    - name: app
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
      resources:
        requests: { cpu: "50m", memory: "64Mi" }
        limits: { cpu: "250m", memory: "256Mi" }
```

5. CIS Kubernetes Benchmark Section

CIS describes the Kubernetes Benchmark as consensus-based secure configuration guidance for Kubernetes. In this lab, CIS Benchmark alignment is used as an assessment reference and learning framework. Because the cluster is a Kind-based private lab rather than a production kubeadm or managed service environment, benchmark results are interpreted with environment context. The expected professional practice is to document PASS, FAIL, WARN, and not-applicable outcomes; assign owners to applicable failures; retest after remediation; and keep benchmark artifacts in the evidence folder.

5.1 Checklist mapping to CIS-style themes

CIS-style Theme	Checklist Coverage
Least privilege	ServiceAccount, Role, RoleBinding, denied secret access, no cluster-wide permissions.
Workload restrictions	Pod Security restricted labels, non-root, no privilege escalation, no privileged pod.
Configuration hygiene	Trivy config scan and manifest review.
Network segmentation	Default-deny NetworkPolicy with explicit allow rules.
Continuous review	Retest commands and benchmark evidence collection.

6. Evidence Highlights

7. How to Use this Checklist

- Use the checklist during every Kubernetes manifest pull request review.
- Treat any privileged container, hostPath mount, missing resource limits, or root user as a blocking issue unless documented as an approved exception.
- Use the evidence column to link every control to a screenshot, report, or repeatable command.
- Store exceptions separately with owner, expiry date, compensating control, and retest plan.
- Promote controls into admission policies once the checklist is stable.

Quality Assurance and Evidence Preservation

QA Item	Required Practice
Evidence naming	Use stable screenshot names and link each item to a control or retest step.
Command repeatability	Store commands in documentation so another reviewer can reproduce validation.
Exception handling	Any accepted risk should include owner, reason, expiry date, and compensating control.
Retest discipline	Do not close a remediation item until the expected negative and positive tests both pass.
CI/CD linkage	Where possible, convert manual validation into security gates or scheduled checks.

Reviewer notes for this Kubernetes security area

This document intentionally combines implementation evidence with risk interpretation. The objective is to show not only that commands were executed, but also why each control matters, how it reduces attack surface, and how the result should be retested after future changes.

References and Standards Basis

Reference	Use in this Document
CIS Kubernetes Benchmark	CIS describes its Kubernetes Benchmark as a product of a community consensus process with secure configuration guidelines for Kubernetes.
Kubernetes RBAC documentation	Kubernetes RBAC uses Role, ClusterRole, RoleBinding, and ClusterRoleBinding objects to manage authorization.
Kubernetes NetworkPolicy documentation	NetworkPolicy governs how pods communicate with each other and with network endpoints.
Kubernetes Pod Security Standards	The restricted profile is intended for strongly hardened workload execution.

Reference URLs used during document preparation: <https://www.cisecurity.org/benchmark/kubernetes>, <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>, <https://kubernetes.io/docs/concepts/services-networking/network-policies/>, and <https://kubernetes.io/docs/concepts/security/pod-security-standards/>.